CNnotator: LLM-Guided Memory Safety Annotation Synthesis

Twain Byrnes*
Carnegie Mellon University
Pittsburgh, PA, USA
binarynewts@cmu.edu

Mike Dodds
Galois, Inc.
Portland, OR, USA
miked@galois.com

Abstract

Memory safety errors account for a large proportion of security bugs in systems written in C; modern languages such as Java and Rust prevent such bugs because they are memory-safe by design. To migrate systems to safer languages or identify memory errors, we must first determine how legacy code manipulates memory. This information is only represented implicitly in such code.

In many cases, memory usage patterns are merely tedious for humans to figure out, rather than truly difficult. In this work, we ask if large language models (LLMs) can perform this task by having them synthesize annotations representing memory usage as specifications in CN, a hybrid testing/verification tool. Our tool, CNnotator, uses LLMs to automatically generate and test CN specifications. We find that current models are able to generate CN specifications for small-to-medium C programs, with the OpenAI o3 reasoning model achieving a 90% success rate on first attempts and 97% overall success, while the chat model GPT-4o correctly annotates 65% of first attempts. These results suggest AI-assisted annotation is becoming practical for real-world C codebases.

CCS Concepts

• **Software and its engineering** → **Software verification and validation**; **Formal software verification**; • **Theory of computation** → *Program specifications*; • **Computing methodologies** → *Artificial intelligence.*

Keywords

memory safety, formal verification, separation logic, large language models, C programming, program annotation

ACM Reference Format:
Twain Byrnes and Mike Dodds. 2026. CNnotator: LLM-Guided Memory Safety Annotation Synthesis. In *1st Workshop on Code Translation, Transformation, and Modernization (ReCode '26), April 12–18, 2026, Rio de Janeiro, Brazil.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3786180.3788313

1 Introduction

Formal methods and AI complement each other well: neural AIs are good at *guessing* while formal methods tools are good at *checking*. In fact, many formal methods problems can be phrased as guess-and-check loops. For example:

Guess:		*Check:*
program	→	specification
type signature	→	type checker
variable assignment	→	SAT solver
proof script	→	ITP tool

Using a trustworthy formal checker, we don't need to trust the guesser. This approach neatly sidesteps the problems of AI unreliability and lack of interpretability. This paper describes *CNnotator*, a tool which targets one such guess-and-check problem: figuring out how C programs use memory.[1]

C and CN. Memory safety issues cause a large proportion of security bugs. For example, Google estimated in 2020 that 70% of Chromium security bugs arose from memory safety errors [7]. In older languages such as C and C++, memory management is tedious to get right, easy to get subtly wrong, and can cause critical security errors that are nearly impossible to detect via unit testing.

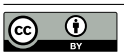
Modern languages, like Java and Rust, have built-in mechanisms that prevent nearly all memory safety bugs, with limited escape hatches for complex memory usage patterns. For users, these mechanisms are mostly automated, and many kinds of memory use can be handled by simple patterns. In C and C++, memory usage patterns cannot be explicitly stated or tested, and therefore they are easy to get wrong. But if we have ways of adding information that can cover simple usage patterns, perhaps we can verify a large amount of C and C++ code memory safe as well?

Rather than annotate the code as is, we might transpile it to a memory-safe language. There are several approaches to this translation problem. Symbolic translation methods, like C2Rust [11], Corrode [21], and Citrus [8], translate C to Rust, but produce unsafe, unidiomatic Rust with the expectation that guaranteed-safe, idiomatic code would require additional manual intervention. On the other hand, AI models can perform transpilation; the CRUST-bench benchmark shows that LLMs can generate idiomatic Rust code they claim to be valid and equivalent [12, 13], but without formal guarantees. C2SaferRust combines symbolic and AI approaches [17]: idiomatic code from the LLM, correctness from the formal methods. All approaches must first understand memory patterns to verify correct translation.

This is one of the biggest challenges in modernizing or translating C code: figuring out how a given program manipulates memory. C programs do many complex things with memory, and this information is only implicit in the code itself. While tedious for humans to extract, these patterns are often simple enough for AI models to infer.

In this project, we built CNnotator, a tool which uses an LLM to determine what individual C functions do with memory. We represent memory usage information using CN [18], a contract language

*Work completed while a Galois intern.



ReCode '26, Rio de Janeiro, Brazil

ACM ISBN 979-8-4007-2411-4
https://doi.org/10.1145/3786180.3788313

[1] Portions of this paper previously appeared on the Galois blog [5].

similar to Frama-C [14], but built on separation logic rather than standard Hoare logic. A contract in CN specifies a precondition and postcondition, and the CN tool then checks whether the contract holds. This testing can be performed by either an SMT solver *or* by automated fuzzing/property-based testing over the contract.

CN has two features that make it particularly useful for our purposes, compared to verification systems such as Frama-C:

(1) *Memory representation.* CN specifications explicitly represent the structure of memory, meaning that specifications represent similar memory usage information to Rust's borrow checker (it inherits these ideas from *separation logic* [20]). In particular, CN enforces memory safety through strict ownership requirements for pointers that are passed around, allocated, and freed. These properties are nearly identical to those that safe Rust enforces by default.
(2) *Automated testing.* A CN specification can be exercised as a runnable test with no further human effort. The Fulminate backend [3] instruments the annotated C code, compiling the specification into runtime checks of internal assertions and postconditions and tracking memory footprints, so testing detects memory-safety violations as well as input-output errors. Building on Fulminate, the Bennet framework [1] derives random input generators directly from the precondition, producing concrete heap states that satisfy it—including for heap-manipulating functions with non-trivial ownership preconditions, where hand-written generators would otherwise be required.

For example, the CN annotations below specify that the function takes *ownership* of the integer at `p` (via `RW<int>(p)`), increments it by one, and returns ownership upon completion. CN can automatically convert this specification into test cases that allocate memory, execute the function, and check that the postconditions hold.

```
void incr(int *p)
/*@ requires take x = RW<int>(p);
    ensures  take y = RW<int>(p);
             y == x + 1i32; @*/
{
  (*p)++;
}
```

There are broadly two kinds of CN annotations: those asserting functional correctness, and those asserting memory safety. In the above example, the first two requirements (those involving read-write permissions) are memory safety annotations. Without the memory safety annotations, CN cannot establish that the code executes without undefined behavior. The last condition asserts functional correctness, describing behavior during normal operation. CNnotator focuses only on memory safety annotations, not functional correctness, which is more subjective.

CN also offers SMT-backed verification, but we used only testing. Each test demonstrates memory safety for that specific run, giving concrete counterexamples on failure while avoiding internal annotations like loop invariants.

CNnotator experiments. We built our prototype tool, CNnotator, to test whether LLMs could generate CN specifications representing memory usage. At a high level, CNnotator follows the guess-and-check pattern:

(1) The AI model *guesses* a possible CN specification for a given C function.
(2) CN's testing backend *checks* that the code passes one hundred automatically generated tests, with Bennet generating concrete heap states from the precondition and Fulminate checking the specification at runtime.

We experimented with several versions of CNnotator. Even the simplest version of this architecture was able to generate specifications for small-to-medium programs, and simple optimization techniques were effective at improving success rates. It seems plausible that, if AI capabilities continue to increase, we will be able to use similar techniques on larger and more complex programs in future.

Paper structure. The remainder of this paper is organized as follows. Section 2 describes the design of CNnotator. Section 3 describes our benchmark suite. Section 4 presents experimental results across five OpenAI models. Section 5 discusses tradeoffs and concludes.

2 CNnotator Tool Design

CNnotator is structured as a loop (Figure 1 shows a simplified workflow):

(1) Pick a C function dealing with memory ownership.
(2) Ask the LLM to generate a CN annotation for that function.
(3) Test the annotation using the CN testing backend.
(4) Depending on the result of testing, either refine the annotation, or go back to step 1 and choose another function.

We implement CNnotator as a command line tool written in Python, with agent control flow and interactions handled by the LangChain and LangGraph libraries [6, 15]. We use the Treesitter library [22] to parse the target C code and extract an abstract syntax tree (AST).

In preprocessing, we collate all C files in the project into one, so the LLM has full context for interdependent functions. We also add the necessary CN libraries and define macros so that allocation and deallocation work correctly with CN.

In step 1, we iterate over the functions in the AST one at a time. We start from the bottom of the file to avoid invalidating AST positions.

In step 2, we construct a prompt for the LLM based on the following template:

CNnotator prompt template:

CN annotation guide. (400 lines approx) Short guide for how to write CN annotations. Since LLMs have not been trained on CN syntax by default, we provide a tutorial to reference. The guide's structure and much of its text is scraped from the CN Tutorial [19], with small additions to cover common failure cases we observed.
Project code. The source code for the whole C project.
Target function. The specific function to annotate.
Formatting instructions. A request to generate JSON output.

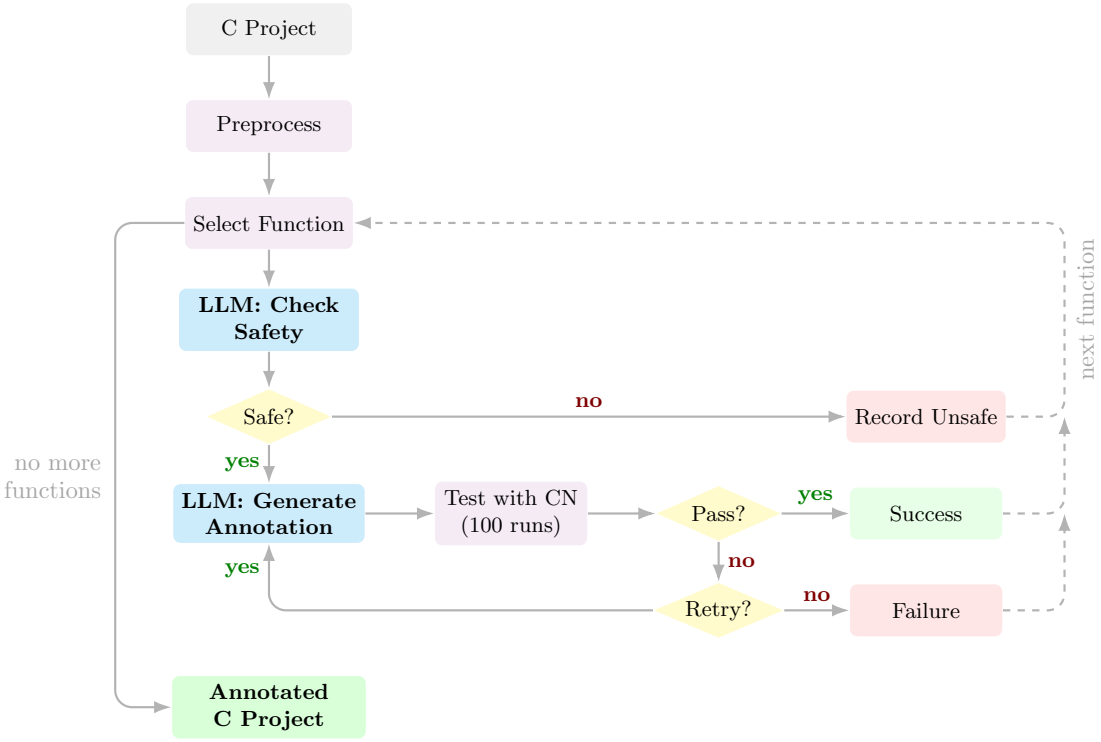


Figure 1: CNnotator workflow. The tool preprocesses C files (merging and adding CN macros), then iterates through each function. For each function, the LLM first checks for obvious memory safety bugs (e.g., use-after-free). If not obviously unsafe, it generates a CN annotation which is tested against 100 generated memory states. Failure tests trigger refinement with error feedback for up to six iterations.

In step 3, we inject the generated annotation into the target C code. We then test the annotation using CN's test synthesis capability. This runs the specification against many generated memory states. We test for 100 valid inputs, which is the default number of runs CN uses for its property-based testing feature.

In step 4, we process the result of running the tests, and choose one of two actions:

(1) If the test passes, we have completed specification generation for this function. We then return to the start of the loop and pick another function.
(2) If it fails, we iterate on the generated annotation. We construct a new prompt consisting of the error, and for some common errors, a hint about how to rectify it. We then test the new annotation again up to n times for any user-specified n. For our evaluation, we chose six, as all of the LLMs we tested seemed to hit diminishing returns around there. Past six, they would either cycle through ideas or progress towards unhelpful ends.

Prior work has shown that prompting an LLM multiple times on the same input leads to higher coverage, scaling with the number of samples [4]. Accordingly, if CNnotator fails to generate an annotation with correct syntax on its first try, it retries up to two more times using the original prompt rather than the error message. We retry with the original prompt because CN syntax error messages are often unhelpful to LLMs: they typically report only "unexpected character" at a location, or rely on visual formatting that models cannot interpret. Empirically, we found that fresh attempts frequently succeed where error-guided iteration does not.

At the end of the loop, CNnotator provides the user with a fully annotated C project, or a comment recording the failure state. CNnotator can fail to generate annotations in three possible situations:

(1) No valid CN annotation exists for the C function, but the code is safe (some safe functions lack valid CN specifications due to the complexities of C memory access);
(2) A valid CN annotation exists, but CNnotator cannot find it;
(3) The C code is inherently unsafe (usually containing a bug), in which case no valid annotation can exist.

CNnotator attempts to distinguish the last case from the first two. After picking a function (Step 1) but before asking the LLM to generate a CN annotation (Step 2), the LLM is asked whether the function is inherently unsafe. If it determines that a function *is* inherently unsafe (e.g., contains a use-after-free or double-free pattern), it inserts a comment declaring that no annotation could ensure safety and records its reasoning in a separate file so the user can see how to modify the original code for safety.[2]

[2]CN is not designed to prove that a function is unsafe, although in future we could ask the LLM to generate such evidence, or reuse CN counterexample traces for this purpose.

3 Benchmark

We created a test suite consisting of 31 annotatable C functions across 28 files and three unannotatable (i.e., inherently unsafe) C functions across three additional files. Of the 31 annotatable functions, 14 were drawn from the CN benchmark suite [19]. We included all of the functions in the benchmark suite that dealt with memory manipulation. These functions test basic CN features without complex memory patterns.

We added 17 harder functions with subtle memory safety requirements, which were generated by Claude Sonnet and o3 as adversarial test cases. We removed comments from these generated functions and introduced additional complexity to challenge the annotation process. The three unannotatable functions contain deliberate memory-safety violations including double-free and use-after-free bugs, and were created manually to test CNnotator's ability to detect inherently unsafe code.

Benchmark Examples. Below is a function from the benchmark set that loops through an array and sets each value equal to 7:

```
void array_3(int *arr, int n) {
  int i = 0;
  while (i < n) {
    *(arr + i) = 7;
    i++;
  };
  return;
}
```

To annotate this function, CNnotator needs to recognize that the passed-in array `int *arr` is accessed for indices up to `n`, and thus must take ownership of array elements `0` through `n-1`.

In the next example, the array is accessed for values up to `ARRAY_SIZE`. CN cannot use macros in annotations, so CNnotator must recognize that `ARRAY_SIZE` is defined at the top of the file and must specify ownership of the first 5 array elements. For the second function, it must make the same decision to replace `ARRAY_SIZE` with 5, but must additionally take ownership of the global variable `globalMultiplier`.

```
#define ARRAY_SIZE 5

int globalMult = 3;

void initGlobalArray(int *globalArray) {
    int i;
    for (i = 0; i < ARRAY_SIZE; i++) {
        *(globalArray + i) = i + 1;
    }
}

void multiplyGlobalArray(int *globalArray) {
    int i;
    for (i = 0; i < ARRAY_SIZE; i++) {
        *(globalArray + i) *= globalMult;
    }
}
```

Test set limitations. We constructed our test set to exercise core CN contract idioms and standard C memory patterns, rather than to represent real-world code at scale. Most examples are under 50 lines, with simple control flow. We did not attempt to exercise the more complex usage patterns that CN supports, such as casts between types and complex pointer arithmetic. Several CN limitations also prevent testing on some categories of production code:

- CN does not support some C types, such as union types.
- Many standard library functions are undefined for CN.
- CN uses custom `alloc` and `free` implementations, with `free` requiring the size of memory being freed. We worked around this using macros rather than allowing CNnotator to modify function bodies.

Despite these limitations, our benchmarks do exercise arrays, loops, global variables, macros, and multi-function files with memory interdependencies between functions.

	Success %	1st Try %	Avg. # Attempts
o3	96.8%	90.3%	1.10
o4-mini	83.9%	74.2%	1.12
GPT-4.1	77.4%	74.2%	1.12
GPT-4o	71.0%	64.5%	1.27
o3-mini	64.5%	61.3%	1.25

Note: Success % shows the percentage of functions successfully annotated. 1st Try % shows functions that succeeded on the first attempt without errors. Avg Attempts is calculated only over successful functions.

Table 1: CNnotator performance across LLM backends.

4 Results

Table 1 presents CNnotator's performance across five LLM backends, while Figure 2 visualizes these results through multiple perspectives: overall success rates (a), first-attempt success without error correction (b), distribution of attempts required (c), and the efficiency frontier (d).

Testing setup. For each benchmark file, we initialized a fresh LLM conversation to prevent information leaking between runs. However, within each file, the LLM maintained its session context, since proper annotations for functions with memory interdependencies may require knowledge of related annotations. CNnotator provides the complete project context to the LLM, enabling it to infer memory relationships and generate better specifications. Table 1 reports only attempts to successfully annotate the annotatable functions; unannotatable functions are excluded from these metrics.

Performance. In Table 1, o3, o3-mini, and o4-mini are reasoning models; GPT-4o and GPT-4.1 are chat models.

CNnotator with o3 achieves the best performance across all metrics in Table 1. It successfully annotates 30 of 31 functions (96.8% overall success), with 28 succeeding on the first attempt (90.3% first-attempt success rate). Only one function remains unannotated after the maximum number of attempts. In other words, CNnotator annotates 90% of the test suite on the first attempt and reaches near-complete coverage with minimal iteration. CNnotator handles both simple ownership patterns and more complex functions with loops, arrays, global variables, and macros.

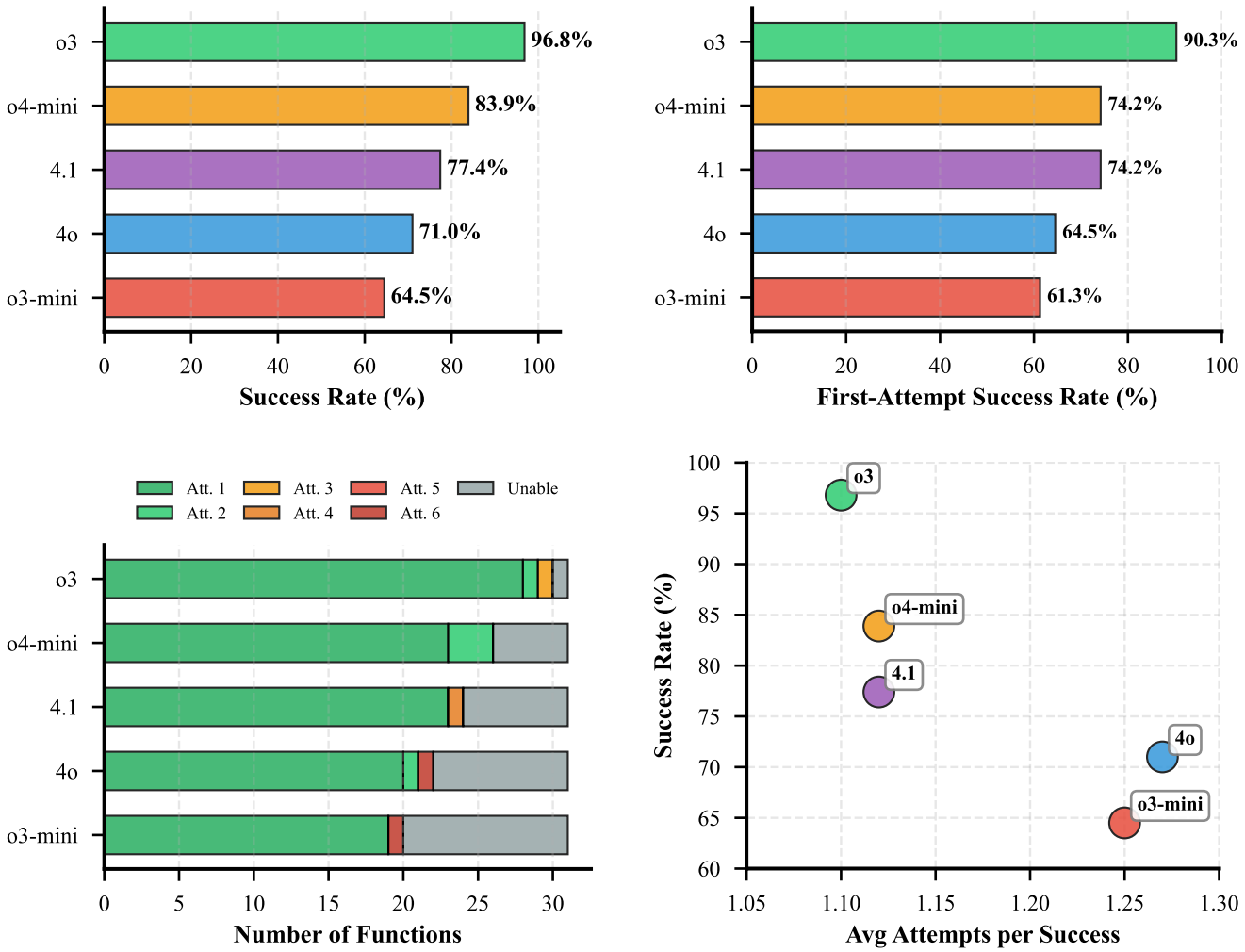


Figure 2: CNnotator performance across five LLM backends on 31 C functions. (a) Overall success rate. (b) First-attempt success rate without error correction. (c) Distribution of functions by attempts required (green: early success, red: multiple attempts or failure). (d) Efficiency frontier showing success rate versus average attempts. o3 achieves the best performance with 96.8% overall success and 90.3% first-attempt success.

All tested GPT models successfully annotated the majority of functions on the first attempt, even chat models not specifically designed for complex reasoning tasks. This aligns with prior research demonstrating LLMs' strong comprehension of C code semantics [10, 16]. Interestingly, GPT-4o (released August 2024, the oldest model tested) annotated 65% of functions on first attempt, outperforming the newer o3-mini and matching o4-mini. This suggests model scale may partially compensate for lacking reasoning-specific optimizations.

As stated in Section 2, if CNnotator's first attempt at an annotation contains incorrect syntax, we retry the initial query up to three times instead of iterating on the broken annotation. We consider a proper annotation on any of these "first attempts" as only taking one attempt, as broken syntax does not yield useful hints. This exploits LLM nondeterminism rather than reasoning.

Failure Modes. There are three failure modes for annotation. The first is inability to generate a passing annotation, which is automatically detected. The second failure mode is the generation of an annotation that passes the round of property-based testing, but the generated specification is still incorrect for some inputs. For our best-performing model (o3), all annotated functions were manually inspected for correctness. For other models, we inspected the adversarial benchmarks, which were more challenging. Manual inspection showed that all annotations deemed correct by CN's property-based testing feature were indeed correct on all inputs.

The third failure mode is that the LLM may "cheat" by generating an over-permissive annotation. For example, if a memory-safe

function is passed a pointer to an array and the length of that array, a CN annotation requiring ownership of twice the length would pass test cases, though it requires more than is necessary and does not match actual usage of the function. Since this kind of pattern depends on the surrounding code, we feed the whole program to CNnotator so that it may make a more informed decision. These usage patterns were manually investigated along with the second failure mode described above. Though different requirements were found across different runs, none violated usage of the function within the file passed in.

Our benchmark includes several inherently unsafe functions containing use-after-free and double-free bugs. All models detected these immediately and declined to annotate them. However, subtler bugs (such as out-of-bounds array access) would be harder to detect. As noted above (failure mode three), CNnotator might generate an over-approximating annotation that requires ownership of more memory than strictly necessary. We consider such over-approximation acceptable: it still yields a valid specification, and the "correct" memory footprint often depends on calling context.

5 Discussion

Framework Tradeoffs. As an experiment, we also built *CNnotatorCC*, reimplementing CNnotator using the agentic framework Claude Code [2]. Because Claude Code edits files directly and manages its own control flow, we built CNnotatorCC in less than a day, compared to several weeks for the original tool. In limited manual testing, CNnotatorCC with Claude Sonnet performed comparably to CNnotator with o3.

However, agentic frameworks require more trust than structured tools. CNnotator inserts annotations via a deterministic Python script; Claude Code may edit code in unexpected ways, including modifying the function under test rather than admitting it cannot find an annotation. We also found Claude Code unreliable at self-reporting: in one case, it reported 3 attempts when logs showed 8, later claiming it had discounted attempts it "deemed unimportant." For unattended automation, structured architectures currently offer stronger guarantees.

Future Work. CNnotator generates memory safety specifications in CN, a separation-logic-based language. This could facilitate direct translation to safe Rust, skipping intermediate unsafe Rust. In future work, we plan to translate C code with CN annotations directly to safe Rust, and evaluate when and how these annotations are helpful. Another potential direction is experimenting with more LLM providers. We expect annotation capabilities to improve over time, as LLM reasoning improves.

Conclusion. CNnotator can help determine whether legacy C code is memory safe, using CN's property-based testing to generate counterexamples that guide annotation refinement. It can synthesize annotations for multi-function, multi-file C projects with various data structures.

Combining LLMs with formal methods enables new approaches to code modernization. As shown by our experiments, LLM outputs are not to be trusted. However, formal checkers can validate them without requiring trust. CNnotator contributes to C-to-Rust translation research, such as DARPA's TRACTOR program [9], by automating memory safety annotation, and CNnotator can also help modernize C code without full translation.

References

[1] Zain K. Aamer and Benjamin C. Pierce. 2025. Bennet: Randomized Specification Testing for Heap-Manipulating Programs. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 3924–3953. doi:10.1145/3764115

[2] Anthropic. 2025. Claude Code: An Agentic CLI Coding Assistant. Anthropic. https://claude.ai/code Accessed 2025.

[3] Rini Banerjee, Kayvan Memarian, Dhruv Makwana, Christopher Pulte, Neel Krishnaswami, and Peter Sewell. 2025. Fulminate: Testing CN Separation-Logic Specifications in C. *Proceedings of the ACM on Programming Languages* 9, POPL, Article 43 (2025), 33 pages. doi:10.1145/3704879

[4] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. 2024. Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787 [cs.LG] https://arxiv.org/abs/2407.21787

[5] Twain Byrnes. 2025. Escaping Isla Nublar: Coming Around to LLMs for Formal Methods. Galois Blog. https://www.galois.com/articles/escaping-isla-nublar-coming-around-to-llms-for-formal-methods Published August 18, 2025.

[6] Harrison Chase. 2022. LangChain. GitHub repository. https://github.com/langchain-ai/langchain Accessed 2025.

[7] Chromium Security Team. 2025. Memory Safety. Chromium Project Documentation. https://www.chromium.org/Home/chromium-security/memory-safety/ Accessed 2025.

[8] Citrus Contributors. 2017. Citrus: C to Rust Converter. GitLab repository. https://gitlab.com/citrus-rs/citrus Accessed 2025.

[9] DARPA Information Innovation Office. 2024. TRACTOR: Translating All C to Rust. DARPA Program Information. https://www.darpa.mil/program/translating-all-c-to-rust Accessed 2025.

[10] Chongzhou Fang, Ning Miao, Shaurya Srivastav, Jialin Liu, Ruoyu Zhang, Ruijie Fang, Asmita, Ryan Tsang, Najmeh Nazari, Han Wang, and Houman Homayoun. 2024. Large Language Models for Code Analysis: Do LLMs Really Do Their Job?. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 829–846. https://www.usenix.org/conference/usenixsecurity24/presentation/fang

[11] Immunant and Galois. 2024. C2Rust: Migrate C Code to Rust. GitHub repository. https://github.com/immunant/c2rust Accessed 2025.

[12] Anirudh Khatry, Robert Zhang, Jia Pan, Ziteng Wang, Qiaochu Chen, Greg Durrett, and Isil Dillig. 2025. CRUST-Bench: A Comprehensive Benchmark for C-to-safe-Rust Transpilation. arXiv:2504.15254 [cs.SE] https://arxiv.org/abs/2504.15254

[13] Anirudh Khatry, Robert Zhang, Jia Pan, Ziteng Wang, Qiaochu Chen, Greg Durrett, and Isil Dillig. 2025. CRUST-Bench Project Website. Project Website. https://crust-bench.github.io Accessed 2025.

[14] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. 2015. Frama-C: A Software Analysis Perspective. *Formal Aspects of Computing* 27, 3 (2015), 573–609. doi:10.1007/s00165-014-0326-7

[15] LangChain Inc. 2024. LangGraph: Build Resilient Language Agents as Graphs. GitHub repository. https://github.com/langchain-ai/langgraph Accessed 2025.

[16] Wei Ma, Shangqing Liu, Zhihao Lin, Wenhan Wang, Qiang Hu, Ye Liu, Cen Zhang, Liming Nie, Li Li, and Yang Liu. 2024. LMs: Understanding Code Syntax and Semantics for Code Analysis. arXiv:2305.12138 [cs.SE] https://arxiv.org/abs/2305.12138

[17] Vikram Nitin, Rahul Krishna, Luiz Lemos do Valle, and Baishakhi Ray. 2025. C2SaferRust: Transforming C Projects into Safer Rust with NeuroSymbolic Techniques. arXiv:2501.14257 [cs.SE] https://arxiv.org/abs/2501.14257

[18] Christopher Pulte, Dhruv C. Makwana, Thomas Sewell, Kayvan Memarian, Peter Sewell, and Neel Krishnaswami. 2023. CN: Verifying Systems C Code with Separation-Logic Refinement Types. *Proceedings of the ACM on Programming Languages* 7, POPL (2023), 191–220. doi:10.1145/3571194

[19] Christopher Pulte, Benjamin C. Pierce, Cole Schlesinger, Jessica Shi, and Elizabeth Austell. 2025. CN Tutorial. Online tutorial. https://rems-project.github.io/cn-tutorial/ Accessed 2025.

[20] John C. Reynolds. 2002. Separation Logic: A Logic for Shared Mutable Data Structures. In *Proceedings of the 17th Annual IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, Copenhagen, Denmark, 55–74. doi:10.1109/LICS.2002.1029817

[21] Jamey Sharp. 2016. Corrode: C to Rust Translator. GitHub repository. https://github.com/jameysharp/corrode Accessed 2025.

[22] Tree-sitter Contributors. 2024. Tree-sitter: An Incremental Parsing System for Programming Tools. GitHub repository. https://github.com/tree-sitter/tree-sitter Accessed 2025.